

Защитная речь — Smart Price (v2, под 16 слайдов, ~10 мин)

Темп: ~140 слов/мин. Текст ~1450 слов = **10 минут 20 секунд**.

Под каждым слайдом — сколько он висит, что говорить дословно, ключевой акцент.

Если поджимает время — сокращаемые фрагменты помечены *[можно опустить]*.

Слайд 1 · Титул (0:00 — 0:15, 15 сек)

Уважаемые члены комиссии, уважаемый председатель! Меня зовут [Фамилия Имя].

Представляю вашему вниманию выпускную квалификационную работу на тему «Разработка веб-платформы AI-метапоиска цен Smart Price». Научный руководитель — [ФИО руководителя]. Тема работы — на экране.

[Пауза 1 секунду, переключаешь на слайд 2.]

Слайд 2 · Актуальность (0:15 — 1:15, 60 сек)

За последние пять лет рынок электронной коммерции России и Беларуси вырос более чем в три раза. Параллельно с этим выросло и количество маркетплейсов — десятки крупных торговых площадок, у каждой свой ассортимент, своя цена, свои акции.

На слайде — реальный пример: один и тот же товар, JBL Flip 7, представлен на пяти площадках с разбросом цены до восьми процентов. Чтобы найти лучшее предложение, пользователь вынужден открыть пять-шесть вкладок, скопировать запрос, дождаться выдачи в каждой и сравнить результаты глазами. По нашим замерам, на это уходит **от восьми до пятнадцати минут**.

Никаких системных решений, которые объединили бы выдачу в едином торговом пространстве России и Беларуси, на момент начала работы не существовало. Зарубежные сервисы — Idealo, PriceRunner — не охватывают локальные маркетплейсы. А отечественные агрегаторы работают по устаревшей модели: периодическая выгрузка прайсов раз в сутки, без AI-обработки и без потокового интерфейса.

[Акцент на «8–15 минут» — это боль, которую решаем.]

Слайд 3 · Цель и задачи (1:15 — 2:15, 60 сек)

Целью работы являлось проектирование и разработка веб-платформы метапоиска товаров на маркетплейсах Беларуси и России с использованием искусственного интеллекта. Область применения — электронная коммерция, веб-приложение с возможным мобильным расширением.

Для достижения цели были поставлены пять основных задач:

Первое — провести анализ предметной области и существующих решений.

Второе — спроектировать архитектуру распределённой системы с поддержкой потоковой передачи данных через Server-Sent Events.

Третье — разработать парсеры для шести маркетплейсов с обходом современных антибот-защит.

Четвёртое — интегрировать AI-ассистента на базе модели Gemini 2.5 Flash через сервис OpenRouter с поддержкой паттерна tool-use.

И пятое — провести комплексное тестирование и развернуть систему в промышленную эксплуатацию.

[Каждую задачу — отдельным предложением, выдерживая короткую паузу. Это даёт ритм.]

Слайд 4 · Решение — общий пайплайн (2:15 — 3:05, 50 сек)

Ключевая идея решения — **параллельный опрос шести независимых источников и потоковая выдача результатов** пользователю по мере готовности каждого из них.

На схеме показан полный пайплайн. Пользователь вводит запрос — backend принимает его, вызывает `asyncio.gather` сразу по шести парсерам, и каждый оффер, как только получен, отдаётся клиенту через Server-Sent Events. Никакой синхронной блокировки — клиент не ждёт самого медленного парсера, он видит результаты сразу.

Параллельно с поиском работает **AI-слой** на базе Gemini 2.5 Flash — он отвечает на вопрос «дорого это или дешево», опираясь на исторический контекст цены.

Слайд 5 · Демо · Поиск (3:05 — 3:35, 30 сек)

На слайде — реальный продакшен-скриншот. Пользователь ввёл просторечный запрос «плойка 5» — система через AI-коррекцию привела его к канонической форме «PlayStation 5», запустила параллельный поиск и нашла **38 предложений из 5 источников**. В шапке видны счётчики по каждому парсеру в реальном времени. Лучшее предложение — Яндекс Маркет, 39 173 рубля.

[Указать пальцем на исправление «плойка 5 → PlayStation 5» — это эффектно демонстрирует AI-коррекцию.]

Слайд 6 · Архитектура (3:35 — 4:45, 70 сек)

Технически система построена на пяти слоях, схема — на экране.

Клиент — Next.js 14 на TypeScript со статической генерацией. Использует EventSource для приёма SSE-потока и Zustand для управления состоянием карточек, появляющихся инкрементально.

Edge — nginx на VPS, проксирует с поддержкой HTTP/2 и ALPN. Критично — отключено `proxy_buffering` для SSE-эндпоинтов, иначе nginx копит ответ и ломает стриминг.

Backend — FastAPI на Python 3.11, асинхронный SQLAlchemy 2.0, библиотека `sse_starlette` для стриминга. Ядро — функция, которая через `asyncio.gather` запускает шесть парсеров и сразу же по мере готовности отдаёт результаты клиенту.

Парсеры и источники — шесть отдельных модулей с единым контрактом `search(query) → list[Offer]`.

[Можно опустить:] Очереди задач — Celery поверх Redis для фоновых работ: обновление истории цен, фоновая нормализация. База — PostgreSQL 16.

Слайд 7 · Парсеры (4:45 — 5:25, 40 сек)

Из двадцати пяти проанализированных маркетплейсов отобрано шесть, по критериям качества данных и стабильности доступа. Это **Onliner.by** — единственный белорусский источник, JSON API. **Яндекс Маркет** — российский, HTML с регулярными выражениями.

Регард и **World Devices** — российский ритейл и импорт электроники. И две недавно подключённые площадки — **1click** и **BigGeek**, специализирующиеся на технике Apple.

Архитектура парсеров — модульная. Новый источник подключается как отдельный async-модуль без изменения ядра.

Слайд 8 · AI-слой (5:25 — 6:05, 40 сек)

В качестве LLM выбран **Gemini 2.5 Flash от Google** через прокси-сервис **OpenRouter**.

Выбор обоснован тремя факторами: высокое качество понимания русского языка, наличие потокового API и стоимость — десять центов за миллион токенов, что на порядок дешевле GPT-4.

Принцип работы — **паттерн tool-use**. Модель сама решает, какие инструменты бэкенда вызвать: получение истории цен, сравнение офферов, расчёт медианы. На вход — текущее предложение, на выход — структурированный вердикт «дорого, норма или дешево» с объяснением.

Слайд 9 · Killer feature — Найти дешевле (6:05 — 7:05, 60 сек)

Это уникальная функция системы, **аналогов которой на рынке нет**.

Идея: товар прогоняется через **закрытый WebSocket-канал бэкенда Яндекс.Алисы** — тот, через который голосовой ассистент находит «то же самое дешевле». Публичного API у этого канала нет, поэтому в рамках работы был выполнен **исследовательский reverse-engineering протокола**.

Алгоритм — три шага. Открываем WebSocket-сессию с UUID-эмуляцией клиента Яндекс. Парсим бинарный payload, извлекая типизированные сущности `EAliceOfferCard`, `EAliceOffer`, `EAliceOfferList`. Мерджим найденные офферы в общую ленту по артикулу или EAN.

Справа на слайде — реальный скриншот: запрос iPhone 17 Pro, лучшая цена 119 090 рублей, что на **16 113 рублей дешевле**, чем на Ozon. То есть фича работает в продакшене и даёт реальный экономический эффект.

[Здесь должна быть гордость в голосе — это центральный аргумент работы.]

Слайд 10 · Функционал (7:05 — 7:45, 40 сек)

За кулисами основного интерфейса работают четыре подсистемы.

AI-коррекция запросов приводит сленг, опечатки и транслитерацию к канонической форме за в среднем 420 миллисекунд, с эффективностью 97 случаев из 100.

Гибридная фильтрация — двухступенчатая. Regex отсекает чехлы и аксессуары, AI-классификатор отделяет основной товар. Релевантность выдачи выросла на 30–40%.

AI-ассистент с tool-use работает с защитой от галлюцинаций — все цифры берутся из реальных вызовов функций бэкенда, а не выдумываются моделью.

И **система истории цен** — динамика за 60 дней по каждому товару, основа для вердикта.

Слайд 11 · Демо · Сравнение и анализ цен (7:45 — 8:15, 30 сек)

Кроме поиска, реализованы ещё две функции — на скриншоте слева **сравнение до четырёх товаров с AI-обоснованием**, справа — **анализ конкретной цены по 100-балльной шкале** относительно медианы рынка с автоматической детекцией региона BY или RU. На примере — iPhone 16 Pro за 79 480 рублей получает 95 баллов из 100, то есть отличное предложение.

Слайд 12 · Метрики (8:15 — 8:45, 30 сек)

Все цифры на слайде — **результат измерений на боевом VPS**, не оценочные расчёты.

Время до первой карточки — **1,4 секунды**. Шесть источников опрашиваются параллельно. Ускорение по сравнению с ручным поиском — порядка ста раз. Объём кода — около пятнадцати тысяч строк на TypeScript и Python. Типовая экономия через функцию Алисы — 10–40 процентов.

Слайд 13 · Экономика (8:45 — 9:25, 40 сек)

Эксплуатация рентабельна. **Стек расходов:** VPS пятнадцать долларов в месяц, OpenRouter ещё пять, домен и SSL — два. Итого **двадцать два доллара в месяц** за полностью работающий сервис.

При средней ставке IT-специалиста пятнадцать долларов в час экономия времени пользователя оценивается в два с половиной — семь с половиной долларов за один поиск. То есть **трёх-девяти использований достаточно, чтобы окупить месяц работы сервиса.**

Возможные модели монетизации — контекстная реклама маркетплейсов, партнёрская программа с CPA-выплатами или премиум-подписка с расширенной аналитикой.

Слайд 14 · Технические вызовы (9:25 — 9:50, 25 сек)

Четыре основных технических вызова, ради которых, по сути, эта работа и существовала. Обход антибот-защит маркетплейсов — решено через фингерпринтинг и ротацию сессий. Геоблокировка Яндекс.Маркета — решена через AI-эвристику по структуре HTML. Реверс бинарного WebSocket-протокола Алисы — решён, фича «Найти дешевле» в проде. И стриминг SSE через nginx — корректно настроен `proxy_buffering off` плюс HTTP/2.

Слайд 15 · Roadmap (9:50 — 10:10, 20 сек)

Дальнейшее развитие — в трёх направлениях. Во втором квартале — мобильное приложение на Expo. В третьем — расширение списка источников: Ozon, AliExpress, DNS и Wildberries — все через готовую плагин-архитектуру. И в четвёртом — модель прогнозирования цены с Telegram-алертами.

Слайд 16 · Финал (10:10 — 10:20, 10 сек)

Работа развёрнута на боевом VPS, доступна по адресу smrt-price.ru и в данный момент обрабатывает реальные пользовательские запросы.

Доклад окончен. Спасибо за внимание. Готов ответить на ваши вопросы.

 Тайм-чарт (для шпаргалки)

№	Слайд	Тайм-код	Длит.	Ключевая мысль
1	Титул	0:00	0:15	Приветствие, тема, руководитель
2	Актуальность	0:15	1:00	8–15 минут на ручной поиск
3	Цель и задачи	1:15	1:00	5 задач, перечисление
4	Решение	2:15	0:50	6 источников + AI-вердикт
5	UI поиск	3:05	0:30	«плойка 5 → PlayStation 5»
6	Архитектура	3:35	1:10	5 слоёв, SSE + asyncio.gather
7	Парсеры	4:45	0:40	25 → 6, модульно
8	AI-слой	5:25	0:40	Gemini 2.5 Flash, tool-use
9	Найти дешевле	6:05	1:00	Главная фишка — реверс WS Алисы
10	Функционал	7:05	0:40	4 подсистемы за кулисами
11	UI сравнение/анализ	7:45	0:30	95/100 баллов из 100
12	Метрики	8:15	0:30	1,4 с, 100×, ~15K строк
13	Экономика	8:45	0:40	\$22/мес, окупаемость 3–9 использований
14	Вызовы	9:25	0:25	4 проблемы решены
15	Roadmap	9:50	0:20	Q2 мобилка, Q3 источники, Q4 прогноз
16	Финал	10:10	0:10	smrt-price.ru, спасибо
Итого			~10:20	

Шпаргалка цифр (для вопросов)

Метрика	Значение
Источников активно	6
Изучено маркетплейсов	25
TTFR (визуальный отклик)	1,4 с
Время до первого батча	5,8 с
Полный поиск	24,2 с
Ускорение vs ручной	~100×
Контейнеров в Docker Compose	6
Стоимость эксплуатации	~\$22/мес
Экономия пользователю	\$2,5–7,5 за поиск
AI-коррекция, время	420 мс
AI-коррекция, точность	97 / 100
Гибридная фильтрация, рост релевантности	+30–40%
Антигаллюцинационные тесты	9 / 10 пройдено
Экономия через Алису	10–40%
Строк кода	~15 000
Объём диплома	100 стр, 18 рис., 23 табл., 25 источн.

Заготовки ответов на вопросы

«Почему SSE, а не WebSocket?»

Серверу нужна только односторонняя передача — поток событий «нашёл, нашёл, нашёл». SSE проще, держит автореконнект из коробки, прозрачно проксируется через nginx и не требует дополнительной библиотеки на фронте. WebSocket был бы избыточен.

«Реверс WebSocket Алисы — это законно?»

Это исследовательский reverse-engineering в рамках академической работы. Условия использования не нарушены — мы работаем с трафиком собственного браузерного клиента, который любой пользователь Яндекс может наблюдать у себя в DevTools. Никакого взлома, никакого доступа к закрытым ресурсам.

«А если Яндекс изменит протокол?»

Это признаваемый риск. Поэтому функция «Найти дешевле» вынесена в отдельный воркер, изолирована от ядра системы. Если протокол меняется — отключается только эта фича, основной метапоиск по шести источникам продолжает работать. На случай изменений предусмотрен мониторинг успешности WS-сессии.

«Почему Gemini, а не GPT-4 или Claude?»

Gemini 2.5 Flash через OpenRouter дешевле GPT-4o Mini примерно в полтора раза и Claude Haiku в два с половиной раза, при сопоставимом качестве для задач нормализации текста и tool-use. Кроме того, OpenRouter решает проблему геоблокировки прямого Gemini API из России. Если бы Gemini перестал устраивать — через OpenRouter можно горячо переключиться на любую другую модель без изменения кода.

«А что с Wildberries и Ozon — почему их нет?»

Они в roadmap на третий квартал. Wildberries был временно отключён из-за нестабильности полей наличия и попадания в выдачу восстановленных товаров вместе с новыми. Ozon — изначально не подключали, потому что у него агрессивная антибот-защита, требующая отдельной инфраструктуры. Архитектура парсеров такова, что новый источник подключается как отдельный модуль — ядро системы менять не нужно.

«Как тестировали 97% точности AI-коррекции?»

Набор из 100 типовых пользовательских запросов с намеренными опечатками, сленгом и транслитерацией — «айфон 16 пра», «сяоми 15», «плойка 5» и так далее. Результат вручную сверялся с эталонной канонической формой. В 97 из 100 случаев модель привела запрос корректно. Подробная таблица — в разделе 4.2.2 диплома.

«Почему именно 6 источников, а не больше?»

Это компромисс между покрытием и стабильностью. Из 25 изученных маркетплейсов далеко не все имеют пригодный для парсинга API или стабильную HTML-структуру. Также важна была разнородность сегментов: ритейл, маркетплейс, нишевые магазины Apple-техники. Шесть источников дают покрытие порядка 80% типовых поисковых сценариев в категориях электроники.

«Какой стек используется на проде?»

Frontend — Next.js 14 на TypeScript, статический экспорт. Backend — FastAPI на Python 3.11 с SQLAlchemy 2.0 async. Очереди — Celery с Redis. База — PostgreSQL 16. Развёртывание — Docker Compose, шесть контейнеров на одном VPS, nginx с HTTP/2 на хосте, не в контейнере.

Финальные напоминания перед защитой

- **Если поджимает время** — пропускай абзацы, помеченные *[можно опустить]*. Это сэкономит ~30 секунд.
- **Если идёшь с запасом** — добавь на слайде 6 фразу про Celery + Redis, на слайде 8 — про конкретные имена моделей.
- **Главные акценты, которые нельзя забыть** — слайды 2 («8–15 минут»), 9 («аналогов на рынке нет»), 12 («измерено в проде»).
- **Если волнуешься — замедлись.** 10 минут — это нормально для защиты, не торопись закончить.
- **На «спасибо за внимание»** — короткая пауза, кивок, и сразу садись. Не топчись у экрана.